

**UNOCHAPECÓ**  
Universidade Comunitária da Região de Chapecó

## **Relatório Técnico**

Trabalho em Grupo: Desenvolvimento Web com CRUD  
*Projeto: Biblioteca Vitória*

Disciplina: Desenvolvimento para WEB  
Curso: Ciência da Computação

Aluno: Alan Fernando  
Chapecó, julho de 2026

# 1. Fase de Planejamento

## 1.1 A empresa fictícia

A Biblioteca Vitória é uma biblioteca comunitária fictícia, fundada em 2011, com sede em Chapecó/SC. Seu acervo reúne mais de 3.000 títulos entre literatura brasileira, obras técnicas e clássicos, com acesso gratuito para a comunidade local. O nicho de mercado escolhido foi o de bibliotecas, por representar bem um caso clássico de gerenciamento de conteúdo (CRUD) aplicado a um catálogo de itens — no caso, livros.

## 1.2 Modelo de dados do CRUD

O CRUD do projeto gerencia a entidade Livro, armazenada na tabela livros do banco de dados PostgreSQL (hospedado no Supabase). A tabela a seguir descreve os campos do modelo:

| Campo                   | Tipo       | Obrigatório      | Descrição                                             |
|-------------------------|------------|------------------|-------------------------------------------------------|
| id                      | uuid       | Sim (gerado)     | Identificador único do livro, gerado automaticamente. |
| título                  | text       | Sim              | Título da obra.                                       |
| autor                   | text       | Sim              | Nome do autor.                                        |
| genero                  | text       | Sim              | Gênero literário (ex.: Romance, Naturalismo).         |
| editora                 | text       | Não              | Editora responsável pela publicação.                  |
| ano_publicacao          | int        | Não              | Ano de publicação da obra.                            |
| isbn                    | text       | Não              | Código ISBN do livro.                                 |
| exemplares              | int        | Sim              | Quantidade de exemplares disponíveis para empréstimo. |
| descricao               | text       | Não              | Sinopse ou descrição da obra.                         |
| capa_url                | text       | Não              | URL da imagem de capa do livro.                       |
| created_at / updated_at | timestampz | Sim (automático) | Datas de criação e última atualização do registro.    |

## 1.3 Wireframes e design geral

O site público foi planejado com quatro páginas principais — Início, Catálogo, Sobre e Contato — acessíveis por uma barra de navegação fixa. A página de Catálogo apresenta os livros em cartões organizados em grade, com um campo de busca por título, autor ou gênero no topo.

O painel administrativo segue um layout de duas colunas: uma barra lateral fixa com os atalhos "Gerenciar Livros", "Novo Livro", "Ver Catálogo Público" e "Sair"; e uma área de conteúdo principal que exibe, conforme a rota, a tabela de livros cadastrados (Read) ou os formulários de cadastro/edição (Create/Update). O acesso a esse painel exige login prévio em /admin/login.

A identidade visual segue uma paleta em tons de marrom escuro e dourado (#1A1408 / #C9A24B) sobre fundo creme (#FAF6EC), com tipografia serifada nos títulos (remetendo a um ambiente de leitura e biblioteca) e tipografia sem serifa no corpo do texto, priorizando legibilidade.

## 2. Desenvolvimento do Front-End

O front-end foi construído com Next.js 14 (App Router) e TypeScript, utilizando Server Components para a renderização das páginas públicas e do painel administrativo, e Client Components apenas onde há interatividade — como no formulário de login e nos formulários de cadastro/edição de livros.

A estilização foi feita inteiramente com Tailwind CSS, o que permitiu construir uma interface responsiva sem escrever CSS customizado: as grades de cartões do catálogo e da página inicial utilizam classes utilitárias de grid que se adaptam automaticamente entre uma coluna (celular), duas colunas (tablet) e três colunas (desktop).

Para usabilidade, a navegação principal permanece fixa no topo da página (sticky), os formulários exibem mensagens de erro contextuais (por exemplo, campos obrigatórios não preenchidos) e os botões de envio mostram um estado de carregamento ("Salvando...") durante as operações assíncronas, usando o hook `useFormStatus` do React.

## 3. Desenvolvimento do Back-End

O back-end foi implementado utilizando o próprio Next.js, por meio de Server Actions — funções que executam no servidor e podem ser chamadas diretamente a partir de formulários React, sem a necessidade de criar endpoints de API manualmente. As quatro operações do CRUD (`criarLivro`, `atualizarLivro`, `excluirLivro` e a leitura via Server Components) estão centralizadas no arquivo `app/admin/livros/actions.ts`.

O banco de dados utilizado é o PostgreSQL, provisionado através da plataforma Supabase, que também fornece o serviço de autenticação (Supabase Auth). A comunicação entre a aplicação e o banco é feita pelo SDK oficial `@supabase/supabase-js`, com dois clientes distintos: um para o navegador (`lib/supabase/client.ts`) e outro para o servidor (`lib/supabase/server.ts`), este último manipulando cookies de sessão via `@supabase/ssr`.

A autenticação do administrador utiliza login por e-mail e senha (Supabase Auth). O acesso às rotas `/admin/*` é protegido por um middleware (`middleware.ts`) que verifica, a cada requisição, se existe uma sessão válida; caso não exista, o usuário é redirecionado automaticamente para a página de login. Como camada adicional de segurança no próprio banco de dados, a tabela `livros` possui políticas de Row Level Security (RLS): qualquer visitante pode ler o catálogo, mas apenas usuários autenticados podem inserir, atualizar ou excluir registros.

## 4. Integração e Desafios

A integração entre front-end e back-end foi facilitada pelo uso do App Router do Next.js, que permite que Server Components busquem dados diretamente do Supabase no servidor (sem necessidade de uma API REST intermediária) e que Server Actions sejam invocadas diretamente pelos formulários do front-end, reduzindo a quantidade de código e pontos de falha na comunicação entre as camadas.

Um dos principais desafios foi garantir que as rotas administrativas fossem protegidas tanto no front-end quanto no banco de dados. A solução adotada combina duas camadas: o middleware do Next.js, que bloqueia o acesso à interface para quem não está autenticado, e as políticas de Row Level Security do PostgreSQL, que bloqueiam a escrita de dados diretamente no banco para qualquer requisição sem sessão válida — mesmo que alguém tentasse contornar o middleware.

Outro desafio foi manter a lista de livros sempre atualizada após uma operação de CRUD sem recarregar a página manualmente. Isso foi resolvido com a função `invalidatePath` do Next.js, chamada ao final de cada Server Action, que invalida o cache das páginas `/admin/dashboard` e `/catalogo` e força a busca de dados atualizados na próxima renderização.

Por fim, como o projeto foi desenvolvido individualmente, também foi necessário planejar cada etapa sequencialmente (modelo de dados → back-end → front-end → autenticação) em vez de dividir o trabalho em frentes paralelas, o que exigiu atenção redobrada ao escopo para manter o projeto enxuto e funcional dentro do prazo.

## 5. Divisão de Tarefas

Este trabalho foi originalmente proposto para ser desenvolvido em grupos de três integrantes, conforme o enunciado da atividade. Por motivos alheios à disciplina, o projeto foi desenvolvido integralmente por um único aluno, responsável por todas as frentes do trabalho:

- Planejamento: definição do nicho, do modelo de dados e do design geral do site e do painel administrativo.
- Front-end: implementação das páginas públicas, do painel administrativo e da responsividade da interface.
- Back-end: modelagem do banco de dados, políticas de segurança (RLS) e implementação das Server Actions do CRUD.
- Autenticação: configuração do login administrativo via Supabase Auth e proteção de rotas via middleware.
- Documentação: elaboração deste relatório técnico e do arquivo README.md do repositório.